PHP · Laravel

Développer une application web MVC, de la première page à la production

Séquence
SQ01

Partie
P00

Niveau
Mastère ESI

Objectifs du module

- Construire une application **MVC** : routes, vues Blade, Eloquent, contrôleurs, formulaires validés
- **Sécuriser** et enrichir : authentification, autorisations, sessions, cache, files d'attente, e-mails
- **Fiabiliser et livrer** : tests automatisés, packages, déploiement vérifié en production

Prérequis : PHP objet, bases HTML et SQL, poste avec PHP 8.3+, Composer et Git.

Trois séquences, 31 heures

Séquence	Contenu	Volume
1 · Fondamentaux MVC et données	Routes, vues, Eloquent, contrôleurs, formulaires	11 h
2 · Sécurité, sessions, asynchrone	Authentification, middlewares, cache, files, e-mails	8 h
3 · Tests, packages, production	Tests des parcours, packages, déploiement	12 h

Un fil rouge : Voyages scolaires

- **Consulter** et gérer les voyages d'un lycée
- **Réserver** la gestion aux enseignants
- **Notifier** les familles, rappeler les échéances
- **Tester** les parcours essentiels, **déployer** une version utilisable

Le même code sert au module **DevOps** : branches, conteneurs, pipeline, supervision.

Comment on travaille

- Chaque intervention : **apport bref**, **activité** sur le fil rouge, **restitution**
- L'IA est **autorisée en usage professionnel** : chaque ligne conservée doit pouvoir être expliquée
- Une ligne que vous ne savez pas expliquer est retirée
- Évaluation finale en intervention 8 : écrit et **validation technique** sur le fil rouge

Pourquoi un framework ?

Besoin	En PHP « nu »	Avec Laravel
Associer une URL à du code	`if` sur `$_SERVER`	`Route::get(...)`
Lire la base	PDO et SQL à la main	Eloquent : `Produit::all()`
Afficher sans faille XSS	`htmlspecialchars` partout	`{{ }}` échappe par défaut
Protéger les formulaires	Jeton CSRF à coder	`@csrf`
Faire évoluer le schéma	Scripts SQL dispersés	Migrations versionnées

Créer le projet

```
composer global require laravel/installer
laravel new voyages-scolaires
cd voyages-scolaires
php artisan serve
```

Ouvrir `http://127.0.0.1:8000` : la page d'accueil de Laravel.

Par défaut, la base est un fichier **SQLite** créé et migré à l'installation.

Où est quoi ?

Dossier ou fichier	Contenu
`routes/web.php`	Les URL de l'application
`app/Http/Controllers/`	Les contrôleurs
`app/Models/`	Les modèles Eloquent
`resources/views/`	Les vues Blade
`database/migrations/`	L'historique du schéma de la base
`.env`	La configuration de **ce** poste

Le fichier .env

```
APP_NAME="Voyages scolaires"
APP_ENV=local
APP_DEBUG=true
APP_URL=http://127.0.0.1:8000

DB_CONNECTION=sqlite
```

- Une configuration **par environnement** : poste, test, production
- Jamais commité : il contient des secrets (`APP_KEY`, mots de passe)
- `APP_DEBUG=true` en production affiche le code et la configuration à tous

Artisan, la ligne de commande du projet

```
php artisan list                    # toutes les commandes
php artisan route:list              # les routes déclarées
php artisan make:controller ...     # générer un contrôleur
php artisan make:model ... -m       # un modèle et sa migration
php artisan migrate                 # appliquer les migrations
php artisan tinker                  # du PHP dans l'application
```

Toujours **à la racine du projet**, là où se trouve le fichier `artisan`.

Une première route

```
// routes/web.php
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('welcome');
});

Route::get('/bonjour', fn () => 'Bonjour Laravel !');
```

Une route associe une **méthode HTTP** et une **URL** à une action.

ACTIVITÉ

À vous

Seul · sur son poste

À produire : le projet qui démarre et une route `/bonjour` qui répond.

1. Créer le projet, le lancer, ouvrir la page d'accueil
2. Ajouter la route `/bonjour`, la vérifier avec `php artisan route:list`
3. Retrouver dans le projet le fichier de la page d'accueil

À RETENIR

À retenir

- Laravel structure le projet : **routes, contrôleurs, modèles, vues, migrations**
- `.env` configure un poste et **ne se commite pas**
- `php artisan` génère le code et administre le projet
- Une route relie une URL à une action

Ensuite : stocker et lire des données avec Eloquent.